

Evaluating Network Topology Tradeoffs in Gem5

SooHyuk Cho
Princeton University
Princeton, NJ, USA
soohyuk.cho@princeton.edu

Daniel Li
Princeton University
Princeton, NJ, USA
dl4247@princeton.edu

Abstract—As core counts increase, the on-chip interconnect becomes a first-order determinant of latency, throughput, and scalability. This report presents a systematic evaluation of seven Network-on-Chip (NoC) topologies in gem5/Garnet: CrossbarGarnet, Mesh_XY, Mesh_westfirst, Pt2Pt, Ring, Tree, and Star. Using a common simulation methodology, we compare these designs across node count, traffic pattern, and offered load, and analyze their behavior using packet latency, accepted traffic, hop count, scalability, and latency decomposition into network and queueing components. Our results show that no single topology dominates across all conditions. CrossbarGarnet and Pt2Pt provide useful upper-bound references for low-hop communication, but their structural cost limits scalability. Mesh-based topologies offer the most robust general-purpose behavior across traffic patterns and node counts. Ring performs well when communication locality is strong, but degrades as path length grows with scale. Tree and Star highlight a different tradeoff: hierarchy can shorten some logical routes, but often introduces stronger traffic concentration near roots or hubs. More broadly, the study shows that accepted-traffic-based saturation and latency decomposition provide clearer insight than raw latency curves alone. Full code is available at <https://github.com/soohyuk-cho/ECE-575-475-Final-Project-NetworkTopology-on-Gem5>.

Index Terms—Network-on-Chip, NoC topology, gem5, Garnet, on-chip interconnect, scalability, latency, throughput

I. INTRODUCTION

As core counts continue to grow, the on-chip interconnect increasingly shapes overall system behavior. A *Network-on-Chip (NoC)* is not just a transport layer between caches, shared resources, and memory controllers; it is a first-order architectural component that affects latency, throughput, congestion behavior, and scalability. Among NoC design choices, topology is one of the most fundamental because it directly determines path length, wiring complexity, router degree, and the network’s ability to sustain load.

Topology selection, however, is rarely straightforward. A design that performs well under one traffic pattern may behave poorly under another. Low-diameter topologies can still suffer from centralized contention, while structurally simple organizations may perform well only when communication is highly local. In practice, these tradeoffs are often discussed qualitatively or evaluated over only a limited set of topologies, node counts, or traffic patterns. Even when simulation-based comparisons are available, conclusions can depend strongly on methodological choices such as how saturation is defined and whether latency is separated into structural traversal cost and congestion-induced queueing.

This report presents a systematic evaluation of NoC topology tradeoffs using gem5’s Garnet interconnect model [1, 2]. We study seven topologies spanning centralized, regular, fully connected, and hierarchical organizations: CrossbarGarnet, Mesh_XY, Mesh_westfirst, Pt2Pt, Ring, Tree, and Star. Our evaluation sweeps node count, traffic pattern, and injection rate, and collects both conventional metrics such as packet latency and hop count and more explanatory metrics such as accepted traffic and latency decomposition. A key methodological decision is to define saturation using peak accepted traffic rather than an arbitrary latency threshold, since baseline latency already differs substantially across topologies.

Our results show that no single topology dominates across all conditions. Mesh-based topologies provide the strongest general-purpose baseline, Ring benefits from localized traffic but becomes less attractive as scale increases, and highly connected designs such as crossbar or point-to-point organizations serve as useful upper-bound references while remaining difficult to scale in practice. Tree- and star-based organizations further expose the tradeoff between short logical paths, centralized bottlenecks, and hierarchical aggregation. More broadly, this work makes three contributions: it develops a reproducible gem5/Garnet-based framework for controlled NoC topology studies, presents a broad comparison across topology classes and traffic conditions under a common methodology, and shows that accepted-traffic-based saturation together with latency decomposition yields clearer conclusions than raw latency curves alone.

II. IMPLEMENTED AND EVALUATED TOPOLOGIES

We study a set of 7 NoC topologies chosen to span several distinct points in the design space: centralized, regular, fully connected, and hierarchical organizations [3–6]. The comparison set includes: CrossbarGarnet, Mesh_XY, Mesh_westfirst, Pt2Pt, Ring, Tree, and Star. Together, these networks let us examine how path length, router degree, and structural bottlenecks affect latency and throughput under different communication patterns [5, 6]. To illustrate the structural differences among the evaluated designs, Fig. 1 shows representative 16-router visualizations for all seven topologies.

CrossbarGarnet represents the most centralized design in the study. Because all communication passes through a single shared router, it minimizes the internal path length,

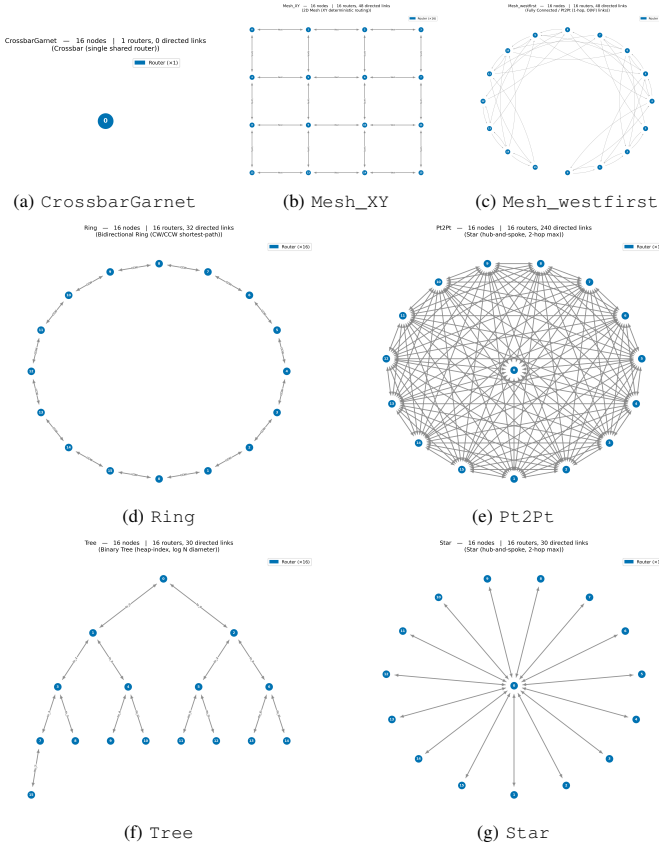


Fig. 1: Representative router-level topology diagrams for the seven evaluated NoC topologies, shown at 16 routers. This node count balances readability with visible structural differences across designs.

but it also concentrates contention and shared resource pressure [5, 6]. As a result, it serves as a useful upper-bound reference for low-hop communication while also illustrating the limits of centralized scaling.

Mesh_XY is the standard deterministic 2D mesh baseline [5–7]. Its main appeal is balance as it avoids the extreme centralization of a crossbar while still keeping path lengths and router degree moderate as the system scales [6, 8]. Mesh_westfirst uses the same physical mesh organization but changes the routing policy, allowing us to isolate how much behavior changes when routing flexibility is introduced without changing the underlying topology [8, 9].

Ring represents a structurally simple topology with low wiring complexity and natural support for local communication [6, 10]. In a bidirectional organization, nearby traffic can travel efficiently in either direction around the ring [10, 11]. However, the topology becomes less attractive as the number of nodes grows as the average distance between communicating endpoints increases more quickly than in a mesh [6, 10]. This makes ring especially useful for contrasting locality-friendly behavior against scalability limits [11].

Pt2Pt (Point-to-Point) is the most highly connected topology in the study [12, 13]. Every source-destination pair has a

direct link [12]. Architecturally, this makes it a valuable reference point because it minimizes path length and represents a practical upper bound on connectivity [12, 13]. At the same time, its router or link complexity grows rapidly with system size, so it is best viewed as a comparison point rather than a scalable deployment target [6, 13].

Finally, Tree and Star introduce hierarchical and hub-based organizations [14–17]. Tree provides a structured multi-level communication pattern that can reduce some long flat connections [14, 15]. Star collapses communication through a central hub [16, 17]. These two topologies are useful because they expose a different set of tradeoffs from mesh and ring: shorter logical routing paths in some cases, but stronger concentration of traffic near upper levels or at the center [14–17].

III. METHODOLOGY

We evaluate all topologies in *gem5* v25.1 using the Ruby/Garnet interconnect model and the synthetic traffic driver `garnet_synth_traffic.py` [1, 2]. The goal is to compare topology behavior under one common setup, so that differences across runs arise primarily from the network organization itself rather than unrelated changes in the simulation environment. The full parameter space is summarized in Table I.

Parameter	Setting
Simulator	gem5 v25.1 with Ruby/Garnet
Traffic driver	<code>garnet_synth_traffic.py</code>
Topologies	CrossbarGarnet, Mesh_XY, Mesh_westfirst, Pt2Pt, Ring, Tree, Star
Node counts	8, 16, 32
Traffic patterns	uniform_random, neighbor, transpose, tornado
Injection rates	0.02–0.50 (15 points)
Simulation length	100,000 cycles/run
Baseline router latency	1 cycle
Baseline VCs per vnet	4
Sensitivity: router latency	1–5 cycles
Sensitivity: VCs per vnet	1, 2, 4, 8

TABLE I: Experimental configuration.

Our core sweep varies node count, traffic pattern, and offered load. We use node counts of 8, 16, and 32; traffic patterns *uniform_random*, *neighbor*, *transpose*, and *tornado*; and 15 injection-rate points from 0.02 to 0.50. Each run executes for 100,000 cycles. Unless otherwise noted, the baseline network uses router latency = 1 cycle and 4 VCs per vnet. A *virtual channel* (VC) is a logical channel multiplexed over a physical

link, used to reduce head-of-line blocking when one packet stalls [6]. A *virtual network* (vnets) is a higher-level traffic class in Garnet, and each vnet is assigned some number of VCs [2]. We also include two targeted sensitivity studies by varying router latency from 1 to 5 cycles and VCs per vnet across 1, 2, 4, and 8.

For each run, we collect standard Garnet statistics including average packet latency, average flit latency, average hops, packet and flit injection/receipt counts, and link-utilization counters [2]. From these, we derive *throughput* and *accepted traffic*, where accepted traffic is delivered flits normalized by simulation length and node count (flits/cycle/node). We also separate flit latency into *network* and *queueing* components. Intuitively, network latency captures router and link traversal, while queueing latency captures waiting for buffers, VCs, or link access. This distinction is important because raw end-to-end latency alone does not reveal whether a topology is limited primarily by path length or by congestion [2, 6].

A key methodological choice is the definition of saturation. Rather than using a heuristic such as the point where latency doubles relative to a low-load baseline, we define saturation as the injection rate at which accepted traffic reaches its peak. This is more stable across topologies because baseline latency can differ substantially even before congestion dominates.

Finally, the *tornado* and *transpose* traffic generators are derived from mesh row/column geometry. They remain useful as common stress patterns, but should be interpreted more cautiously for non-mesh topologies than for mesh-based ones.

IV. RESULTS

We present results across *six* dimensions: zero-load structure, latency under load, saturation throughput, scalability, latency decomposition, and sensitivity to microarchitectural parameters. Unless stated otherwise, figures use 16- or 32-node configurations, which best expose topology-level differences. 8-node results are consistent but compress the signal.

A. Zero-Load Structure: Hops and Low-Load Latency

Before examining load-dependent behavior, we establish the structural baseline using average hop count and zero-latency (injection rate 0.02, *uniform_random*).

Hop count. At 16 nodes, *CrossbarGarnet* records zero internal hops by construction, while *Pt2Pt* averages just under one hop. The measured *Pt2Pt* value is slightly below the theoretical 1.00 because some packets terminate at controllers attached to the same router and therefore incur zero internal hops. *Star* averages 1.76 hops, reflecting a mix of one-hop hub-adjacent traffic and two-hop spoke-to-spoke traffic. *Mesh_XY* and *Mesh_westfirst* both sit near 2.0 hops, consistent with the expected average path length of a 4×4 mesh. *Ring* rises to 2.83 hops, already above the mesh at the same node count, while *Tree* is deepest at 3.25 hops because leaf-to-leaf communication frequently traverses higher levels of the hierarchy.

Zero-load latency. Hop count alone does not fully predict zero-load latency, as Table II shows. *CrossbarGarnet*

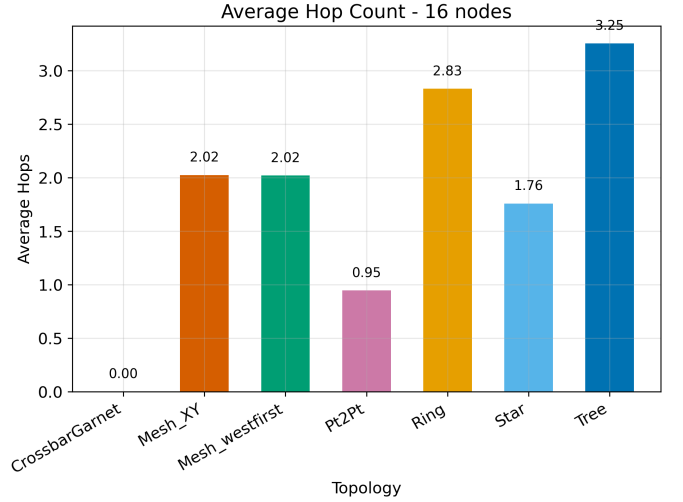


Fig. 2: Average hop count per topology at 16 nodes, measured at the lowest injection rate (0.02) and averaged across traffic patterns. Hop count reflects pure structural path length before any queueing effects appear.

Topology	8 nodes	16 nodes	32 nodes
CrossbarGarnet	4,908	5,042	4,693
Pt2Pt	5,486	5,667	5,522
Star	6,125	6,508	6,474
Mesh_XY	6,542	7,100	8,010
Mesh_westfirst	6,611	7,146	8,029
Tree	6,708	8,191	8,705
Ring	7,125	8,944	11,548

TABLE II: Zero-load packet latency (ticks) at injection rate 0.02, *uniform_random* traffic, by topology and node count.

achieves the lowest latency at all scales, while *Pt2Pt*, despite its near-minimal hop count, remains roughly 500–900 ticks higher. This gap is consistent with router-radix overhead: *Pt2Pt*’s high-degree routers add buffering and arbitration cost that offset its path-length advantage. A similar effect appears in *Star*, which stays well below *Ring* in zero-load latency despite not always having fewer hops. As scale increases, *Ring* shows the strongest growth, rising from 7,125 ticks at 8 nodes to 11,548 ticks at 32 nodes under *uniform_random* traffic, whereas *Mesh_XY* grows more modestly from 6,542 to 8,010 ticks over the same range. *Tree* remains between *Ring* and the mesh topologies across all tested node counts.

B. Latency vs. Injection Rate

Figure 3 shows the latency curves under *uniform_random* traffic at 16 nodes. *CrossbarGarnet* maintains the lowest latency across the full injection range, with *Pt2Pt* close behind. The two mesh variants cluster tightly and are effectively indistinguishable at this scale. *Ring* is the clear high-latency outlier: it begins from a high baseline and flattens early, consistent with earlier saturation. *Tree* follows a similar trend, though slightly below *Ring* at low load and with a steeper rise at moderate load, which is consistent with growing concentration near the root. *Star* stays between the

mesh topologies and the more path- or hub-limited designs, forming an intermediate point between regular and centralized organizations.

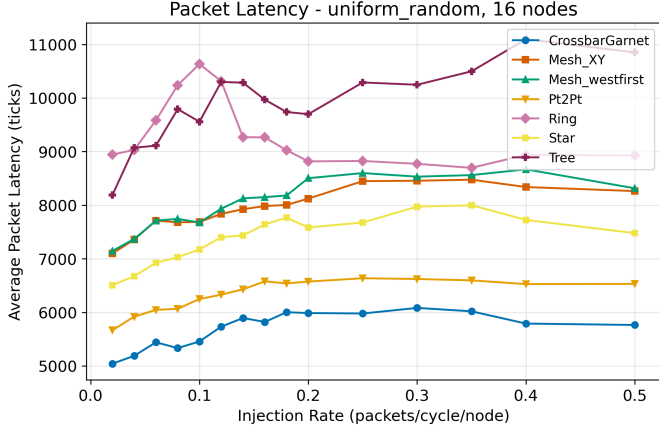


Fig. 3: Average packet latency vs. injection rate under **uniform_random** traffic at 16 nodes. Ring’s higher base latency (driven by longer average paths) is visible even at the lowest injection rate. CrossbarGarnet and Pt2Pt represent low-hop upper-bound references.

Figure 4 tells a different story. Under *neighbor* traffic at 32 nodes, Ring becomes the second-best topology after CrossbarGarnet, outperforming both mesh variants at low and moderate load. The reason is direct: in a bidirectional ring, adjacent communication maps to very short CW/CCW paths. Star also benefits under this pattern because spoke-to-spoke traffic routes through the hub in a fixed number of hops. Tree is the worst performer, since even nearby logical neighbors may need to communicate through higher ancestors before returning to the destination subtree.

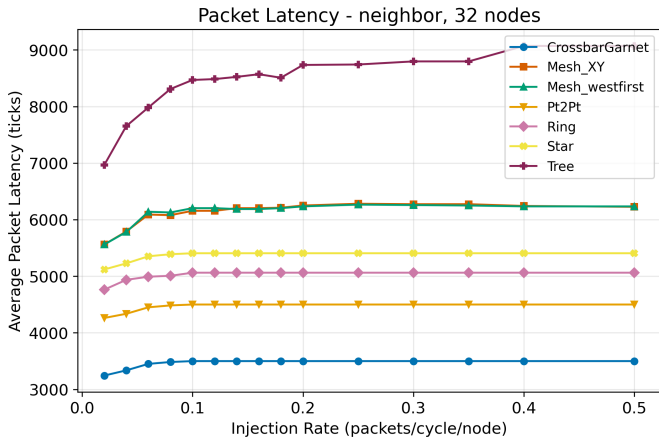


Fig. 4: Average packet latency vs. injection rate under **neighbor** traffic at 32 nodes. Ring performs substantially better than the meshes here because adjacent communication maps to short CW or CCW paths. Tree is the worst-performing topology as leaf-to-leaf neighbor traffic must traverse the root.

These two plots together illustrate a recurring theme: the ranking of topologies is not stable across traffic patterns, and a design’s fit to the communication pattern matters as much as its raw diameter.

C. Saturation Throughput

We define *saturation* as the injection rate at which accepted traffic—flits delivered per cycle per node—reaches its maximum. This definition is more stable than a latency-inflation threshold because baseline latency already differs substantially across topologies. Figure 5 shows the resulting saturation throughput at 32 nodes.

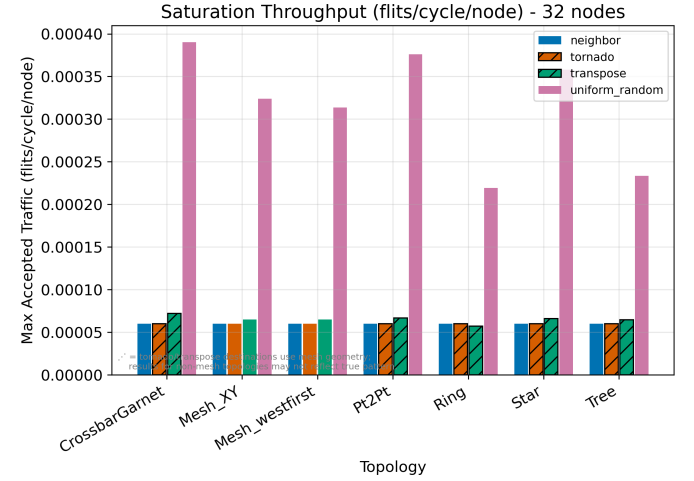


Fig. 5: Peak accepted traffic (flits/cycle/node) at 32 nodes across topologies and traffic patterns. Hatched bars indicate tornado and transpose results for non-mesh topologies, where the destination formula uses mesh geometry and results should be interpreted cautiously. Uniform_random (pink) provides the cleanest comparison.

Under *uniform_random* traffic, which provides the cleanest comparison, CrossbarGarnet achieves the highest peak accepted traffic, followed closely by Pt2Pt and then Star. These three designs share a common structural feature: centralized or high-degree routing resources that move cross-network traffic in few hops before congestion builds up. The two mesh variants form a second tier and remain very close to one another, while Ring and Tree achieve the lowest peak throughput. For Ring, this behavior is consistent with limited bisection bandwidth under global traffic, since cross-ring communication must pass through a small number of critical links.

Between the two mesh-based designs, Mesh_westfirst does not materially improve over Mesh_XY under any tested traffic pattern or node count. Their peak throughputs differ only marginally, and their latency curves are nearly identical. Under these synthetic traffic patterns, physical topology has a much stronger effect on performance than the added routing flexibility of west-first routing.

D. Scalability

Figure 6 plots latency at a moderate injection rate (0.10) as node count increases from 8 to 32. The trends are consistent with theoretical diameter arguments.

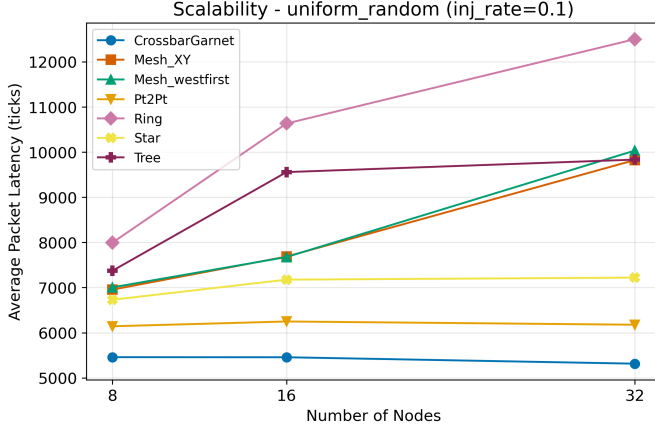


Fig. 6: Average packet latency at injection rate 0.10 vs. node count under uniform_random traffic. Ring’s latency grows steeply with scale while CrossbarGarnet and Pt2Pt remain nearly flat. Mesh topologies scale gracefully.

CrossbarGarnet and Pt2Pt remain nearly flat across the full range because their effective path length does not grow with system size. Both therefore serve as useful reference points, although their structural cost scales poorly: CrossbarGarnet requires increasing router radix, while Pt2Pt requires quadratic link growth. Star also scales gently because communication remains bounded by the hub, although the hub itself becomes increasingly loaded.

The mesh variants grow more moderately and preserve their role as practical scalable baselines. Mesh_XY rises from 6,955 ticks at 8 nodes to 9,825 ticks at 32 nodes, and Mesh_westfirst follows the same trend. Ring pays the steepest penalty among the regular topologies, rising from 7,995 to 12,503 ticks over the same range. In relative terms, the gap between Ring and Mesh_XY widens with scale, which is consistent with the faster growth of path length in the ring. Tree lies between these extremes: hierarchy limits path growth compared to Ring, but concentration near upper levels prevents it from scaling as smoothly as the mesh.

E. Latency Decomposition: Network vs. Queuing

Figure 7 separates flit latency into its network component (time spent traversing routers and links) and queueing component (time waiting for VC allocation or link access). This decomposition answers a question that raw latency curves cannot: is a topology’s high latency caused by long paths, or by congestion at bottleneck resources?

At low-to-moderate load (injection rate 0.10). The network component dominates for Ring and Tree, which exhibit queuing fractions of only 28% and 29% respectively. Their high absolute latency (10k and 9.3k ticks) is almost

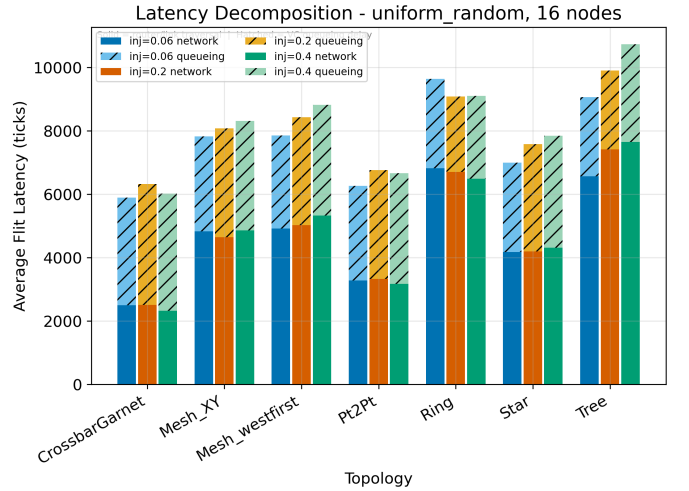


Fig. 7: Flit latency decomposed into network component (router and link traversal; solid) and queueing component (VC wait time; hatched) at 16 nodes under uniform_random traffic, at injection rates 0.05, 0.20, and 0.40. Topologies with high queueing fractions are congestion-limited while those with high network fractions are path-length-limited.

entirely structural. Packets spend most of their time actually traversing routes and links, not waiting in queues. This means their latency cannot be improved substantially by adding buffering of more VCs as the bottleneck is path length.

Conversely, CrossbarGarnet’s queueing fraction is 60%: only 2,336 ticks are attributable to traversal, while 3,471 ticks are spent waiting. This reflects centralized contention at the single shared router. Pt2Pt follows at 52% queueing, because its high-radix router must arbitrate among $N-1$ input ports. Star sits at 43%, inheriting a similar hub-contention dynamic at smaller scale. The two mesh variants both hover around 41%, a balanced regime where neither traversal nor queueing fully dominates.

Near saturation (injection rate 0.40). Strikingly, the decomposition fractions remain nearly stable across injection rates for most topologies. For example, CrossbarGarnet’s queueing fraction moves from 60% at injection 0.10 to 62% at 0.40. Similarly, Ring’s stays at 28-29%. This stability indicates that these topologies do not shift qualitatively between operating regimes within the tested injection range: topologies that are congestion-limited stay congestion-limited, and those that are path-limited stay path-limited.

The implication for such result is clear. For Ring and Tree, the most effective improvement would be structural properties such as shorter physical paths, fewer hops, or hierarchical aggregation. For CrossbarGarnet and Pt2Pt, the bottleneck is resource contention, and additional buffering or VC counts would provide more relief.

F. Sensitivity to Microarchitectural Parameters

The sensitivity studies use Mesh_XY at 16 nodes under uniform_random traffic as a representative baseline. Results

are shown in Figures 8 and 9.

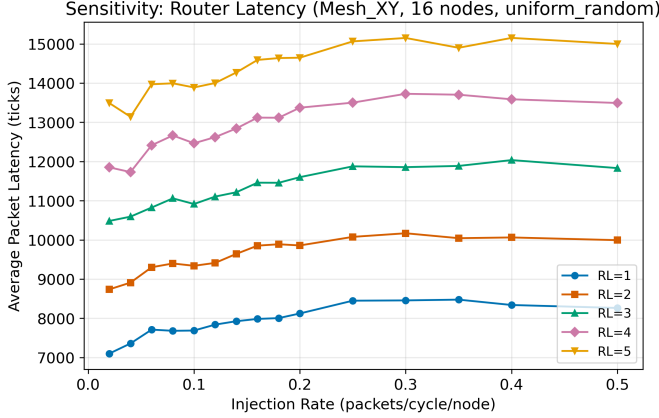


Fig. 8: Packet latency vs. injection rate on Mesh_XY (16 nodes, uniform_random) for router latencies 1-5 cycles. Increasing router latency shifts curves upward nearly uniformly, with proportional effects at both low and high load.

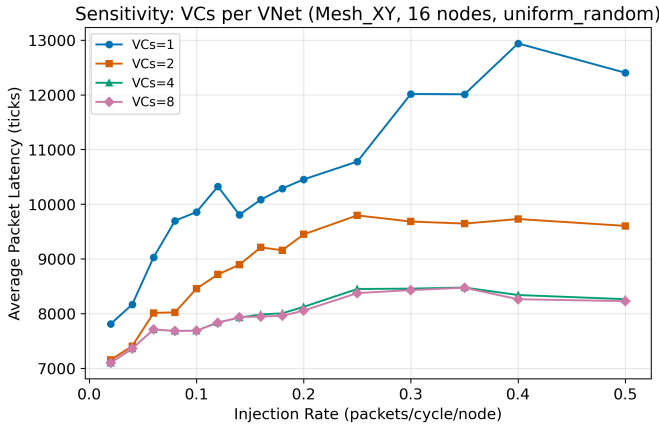


Fig. 9: Packet latency vs. injection rate on Mesh_XY (16 nodes, uniform_random) for 1, 2, 4, and 8 VCs per vnet. VC=1 causes severe head-of-line blocking at moderate load. Returns diminish sharply beyond 4 VCs.

Increasing router latency from 1 to 5 cycles shifts the full latency curve upward at both low and high load, but does not substantially change the qualitative shape of the curve. This suggests that moderate increases in router delay affect absolute latency more than topology-level behavior. Varying the number of VCs per vnet shows a clearer threshold effect: moving from 1 to 2 VCs recovers most of the performance lost to head-of-line blocking, while moving from 4 to 8 VCs provides little additional benefit. The baseline choice of 4 VCs is therefore well placed for the traffic conditions studied here.

V. DISCUSSIONS AND LIMITATIONS

The results show that no single topology dominates across all operating conditions. Instead, each topology occupies a

different part of the design space, and the best choice depends strongly on the traffic pattern and the scaling target. CrossbarGarnet and Pt2Pt provide useful upper-bound references for low-hop communication, but their advantages come from structural assumptions that are difficult to scale in practice. Mesh_XY and Mesh_westfirst offer the most robust general-purpose behavior across node counts and traffic patterns, while Ring becomes attractive only when communication locality is strong. Tree and Star further show that hierarchical organization can shorten some logical routes, but often at the cost of stronger traffic concentration near roots or hubs.

Design Implications. It's important to highlight a few design implications followed from this comparison. First, low hop count alone is not sufficient to predict good performance. Pt2Pt is a clear example for this as it does not consistently outperform CrossbarGarnet in latency despite its one-hop structure. Additionally, its advantage over the mesh topologies is smaller than hop count alone would suggest. Similarly, Star can achieve moderate path lengths while still paying a hub-contention penalty. These cases reinforce that topology evaluation should account not only for geometric distance, but also for where traffic and arbitration pressure accumulate.

Second topology ranking is highly traffic-dependent. Under uniform random traffic, centralized or highly connected designs sustain the highest accepted load, while Ring and Tree saturate earlier because load concentrates on a small set of structural bottlenecks. Under neighbor traffic, however, Ring improves substantially because adjacent communication maps well to short CW/CCW routes. This shift in ranking is one of the main conclusions of the study: topology should be judged relative to the communication pattern it is expected to serve, not only by its worst-case diameter or by a single aggregate metric.

Third, the latency-decomposition results provide a more informative explanation than latency curves alone. Ring and Tree are primarily path-limited over the tested range since their latency is dominated by traversal rather than waiting. CrossbarGarnet and Pt2Pt, in contrast, are more congestion-limited, with a larger fraction of latency coming from queueing and arbitration. This distinction is useful because it separates two different optimization directions. A path-limited topology is unlikely to improve substantially through additional buffering alone, whereas a congestion-limited topology may benefit more from microarchitectural changes in arbitration, buffering, or channel allocation.

The comparison between Mesh_XY and Mesh_westfirst is also instructive. Across the tested traffic patterns and node counts, west-first routing provides little measurable advantage over deterministic XY routing. This does not imply that adaptive routing is ineffective in general. Rather, it suggests that under the Garnet synthetic traffic patterns used here, the physical topology plays a larger role than routing flexibility in determining overall behavior. In this setting, the regular mesh structure dominates the routing policy.

Limitations. Several limitations should be stated clearly. First, the study uses synthetic traffic rather than full application traces. This is appropriate for controlled topology comparison, but it does not capture all locality and burstiness patterns of real workloads. Second, the tornado and transpose generators are derived from mesh row/column geometry, so their interpretation is less direct for non-mesh topologies than for mesh-based ones. Third, the sensitivity studies are limited to `Mesh_XY` and should therefore be interpreted as a methodological sanity check rather than a universal statement about all topologies. Finally, the present study focuses on performance-oriented metrics such as latency, throughput, and scalability. Physical design costs such as area, wiring complexity, power, and floorplanning difficulty are discussed qualitatively but are not modeled directly.

Overall, the main lesson is that topology choice should be guided by expected communication structure and scaling goals rather than by a single abstract notion of connectivity. If the target workload exhibits strong locality, `Ring` can be highly competitive despite its poor large-scale diameter. If robust behavior across diverse traffic patterns is the priority, the mesh topologies remain the strongest practical baseline. `CrossbarGarnet` and `Pt2Pt` are valuable as reference points, but their structural cost makes them better upper bounds than deployment targets. `Tree` and `Star` occupy an intermediate region, highlighting how hierarchy can help path length while simultaneously introducing new concentration bottlenecks.

VI. RELATED WORKS

Network-on-Chip research has long established that on-chip interconnect is a first-order architectural design problem rather than a secondary wiring concern. Early work argued for packet-switched on-chip communication as a scalable alternative to ad-hoc global interconnects, while subsequent surveys and textbooks organized the design space in terms of topology, routing, flow control, and physical cost [3–6]. Our work builds on this foundation, but focuses on a narrower questions: how representative NoC topologies compare under a single, controlled `gem5` and `Garnet`-based methodology.

Topology Families and Routing Policies. A large body of prior work has studied specific topology families and routing policies. `Mesh` networks remain the most common baseline because they offer a practical balance between scalability, router degree and path length [6, 7]. Adaptive routing policies such as west-first have also been studied extensively as a way to improve load balance without changing the underlying network structure [8, 9]. Other works have examined alternative organizations including ring-based interconnects, hierarchical rings, fat-tree designs, and star-like communication structures, often showing that each of these topologies performs well only in certain traffic or scaling regimes [10, 11, 14–17]. Point-to-point organizations have similarly been used as idealized upper-bound references because they minimize path length at the cost of rapidly growing structural complexity [12, 13]. Our evaluation is complementary to these studies: rather than

proposing a new topology or routing algorithm, we compare several of these representative organizations within one common experimental framework.

Simulator Infrastructure for NoC Evaluation. This work is also related to prior simulator infrastructure for NoC evaluation. `Garnet` provides a detailed on-chip network model inside `gem5`, making it possible to study latency, routing, and congestion behavior within a timing-aware architectural simulator [1, 2]. That level of detail is useful for our study because the goal is not only to compare raw throughput or hop count, but also to analyze where latency arises and how topology interacts with queueing and saturation. In this sense, our contribution is not a new NoC simulator, but a systematic comparative study built on an existing full-system simulation substrate.

Relative to prior work, the closest position of this paper is as a controlled comparative study. The novelty is not in introducing another network organization, but in evaluating centralized, regular, fully connected, and hierarchical topologies side by side under one methodology, using accepted-traffic-based saturation and latency decomposition to distinguish structural path cost from congestion-driven delay.

VII. CONCLUSIONS

This paper presented a systematic evaluation of seven NoC topologies in `gem5/Garnet`: `CrossbarGarnet`, `Mesh_XY`, `Mesh_westfirst`, `Pt2Pt`, `Ring`, `Tree`, and `Star`. Using a centralized experimental setup, we compared these designs across node count, traffic pattern, and offered load, and analyzed their behavior using latency, accepted traffic, scalability, and latency decomposition.

Our results show that topology choice is strongly workload-dependent. `CrossbarGarnet` and `Pt2Pt` provide useful upper-bound references for low-hop communication, but their structural cost limits scalability. `Mesh`-based topologies offer the most robust general-purpose behavior across traffic patterns and scales. `Ring` performs well when communication locality is strong but degrades as path length grows with system size. `Tree` and `Star` highlight a different tradeoff: hierarchy can shorten some logical routes but often introduces stronger traffic concentration near roots or hubs. More broadly, the results show that hop count alone is not sufficient to explain performance; separating network latency from queueing latency provides a clearer view of whether a topology is primarily path-limited or congestion-limited.

There are several natural next steps. One is to extend the study beyond synthetic traffic and evaluate topology behavior under application-driven communication patterns. Another is to incorporate physical design considerations such as area, wiring complexity, and power more directly into the comparison. Together, these extensions would help connect topology-level simulation results more closely to practical NoC design decisions.

REFERENCES

- [1] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *SIGARCH Comput. Archit. News*, vol. 39, p. 1–7, Aug. 2011.
- [2] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "Garnet: A detailed on-chip network model inside a full-system simulator," in *2009 IEEE International Symposium on Performance Analysis of Systems and Software*, pp. 33–42, 2009.
- [3] L. Benini and G. De Micheli, "Networks on chips: A new soc paradigm," *Computer*, vol. 35, p. 70–78, Jan. 2002.
- [4] W. J. Dally and B. Towles, "Route packets, not wires: on-chip interconnection networks," in *Proceedings of the 38th Annual Design Automation Conference, DAC '01*, (New York, NY, USA), p. 684–689, Association for Computing Machinery, 2001.
- [5] T. Bjerregaard and S. Mahadevan, "A survey of research and practices of network-on-chip," *ACM Computing Surveys*, vol. 38, no. 1, pp. 1–51, 2006.
- [6] W. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. The Morgan Kaufmann Series in Computer Architecture and Design, Elsevier Science, 2004.
- [7] B. Grot and S. W. Keckler, "Scalable on-chip interconnect topologies," in *2nd Workshop on Chip Multiprocessor Memory Systems and Interconnects (CMP-MSI), 2008*, 2008.
- [8] A. V. De Mello *et al.*, "Evaluation of routing algorithms on mesh based nocs," tech. rep., PUCRS (Pontifical Catholic University of Rio Grande do Sul), Porto Alegre, Brazil, 2004.
- [9] C. J. Glass and L. M. Ni, "The turn model for adaptive routing," *J. ACM*, vol. 41, p. 874–902, Sept. 1994.
- [10] J. Kim and H. Kim, "Router microarchitecture and scalability of ring topology in on-chip networks," in *Proceedings of the 2nd International Workshop on Network on Chip Architectures, NoCArc '09*, (New York, NY, USA), p. 5–10, Association for Computing Machinery, 2009.
- [11] C. Fallin, X. Yu, G. Nazario, and O. Mutlu, "A High-Performance Hierarchical Ring On-Chip Interconnect with Low-Cost Routers," 6 2018.
- [12] Y. Zhang, B. Ozisikyilmaz, G. Memik, J. Kim, and A. Choudhary, "Analyzing the impact of on-chip network traffic on program phases for cmps," in *2009 IEEE International Symposium on Performance Analysis of Systems and Software*, pp. 218–226, 2009.
- [13] D. Vainbrand and R. Ginosar, "Network-on-chip architectures for neural networks," in *Proceedings of the 2010 Fourth ACM/IEEE International Symposium on Networks-on-Chip, NOCS '10*, (USA), p. 135–144, IEEE Computer Society, 2010.
- [14] D. Ludovici, F. Gilabert, S. Medardoni, C. Gómez, M. E. Gómez, P. López, G. N. Gaydadjiev, and D. Bertozzi, "Assessing fat-tree topologies for regular network-on-chip design under nanoscale technology constraints," in *Proceedings of the Conference on Design, Automation and Test in Europe, DATE '09*, (Leuven, BEL), p. 562–565, European Design and Automation Association, 2009.
- [15] A. Bouhraoua and M. E. Elrabaa, "An efficient network-on-chip architecture based on the fat-tree (ft) topology," in *2006 International Conference on Microelectronics*, pp. 28–31, 2006.
- [16] W. Shi and P. K. Srimani, "Hierarchical star: a new two level interconnection network," *J. Syst. Archit.*, vol. 51, no. 1, pp. 1–14, 2005.
- [17] J.-Y. Kim, J. Park, S. Lee, M. Kim, J. Oh, and H.-J. Yoo, "A 118.4 gb/s multi-casting network-on-chip with hierarchical star-ring combined topology for real-time object recognition," *IEEE Journal of Solid-State Circuits*, vol. 45, no. 7, pp. 1399–1409, 2010.